

Beyond Boolean Intersections: A Verified Bisimulation and Trace Decompiler for Causal Graphs

LUCIA YIZI ZHANG, Shanghai Lixin University of Accounting and Finance, China

We mechanize a corrected equivalence between two standard characterizations of d-separation in Lean 4, exhibit a concrete counterexample to the unrestricted statement, and build a witness extractor from graph reachability.

The two characterizations—trail blocking [7] and moralized ancestral-graph separation [4]—have been treated as unconditionally equivalent in the literature. We machine-check a concrete counterexample. It shows that the equivalence fails when an endpoint node lies in the conditioning set Z ; we identify the repair ($\text{DisjointSets } X \ Y \ Z$) and prove the corrected equivalence.

The forward direction compiles any active trail through a Bayes-ball state machine into a compressed moral-graph walk. The reverse direction reconstructs an explicit active-trail witness from moral-graph reachability, including a proved bad-collider normalization (`route_improves_of_bad`) that decreases a well-founded measure. Together, both directions yield `dSeparated_iff_dSeparates` on the pairwise-disjoint domain.

The graph result supports a finite Shannon-leakage bound. We bridge to operational QIF through a two-stage factorisation split: an upper computable topological layer producing decide-able Markov-blanket and local-Markov boundary conditions, and a lower typed adapter that turns those node-set premises into the probabilistic conditional-independence predicate `condMarkov`. Composing the split with a dual KL witness yields an upper bound for information leakage from structural d-separation to quantitative capacity.

1 Introduction

Formalizing both standard characterizations of d-separation in Lean 4’s dependent type system yields a mechanized counterexample: the unrestricted equivalence between Pearl’s trail-blocking predicate and Lauritzen’s moralized ancestral-graph separation is *false*. When an endpoint node lies in the conditioning set Z , the moral-graph construction deletes that endpoint while the trail predicate still admits the one-edge trail that has no internal triple.

The counterexample (Section 2.3) lives on a three-node chain and exhibits the disagreement on a single endpoint configuration.

We mechanize this discrepancy and its repair, treating the two characterizations not as interchangeable definitions but as a checked equivalence between two semantic layers:

- an **operational semantics** of active trails, with `DisjointSets` as the disjointness side condition on the conditioning set;
- a **static analysis** of moralized-graph reachability, with the collider `MAGWalk.jump` as the compression step;
- a **witness extractor** from static reachability back to an operational witness, i.e. a Bayes-ball `StaticRoute` $\rightarrow$ `OpenTrace` $\rightarrow$ `ActiveRoute` pipeline.

The forward direction is a soundness result: it compiles an operational trace into static reachability evidence. The reverse direction reconstructs a concrete execution witness from static reachability alone. The counterexample shows that the two languages are not equivalent without the disjointness side condition.

1.1 Contributions

- (1) We machine-check a counterexample. It shows that unrestricted trail blocking and moral-graph separation are not equivalent; the repair is `DisjointSets X Y Z`.

- (2) We formalize both d-separation characterizations in Lean 4 as a three-layer stack: graph syntax, certified intermediate representations (MAGWalk, StaticRoute, OpenTrace, ActiveRoute), and witness-extraction pipelines.
- (3) We mechanize the forward compiler (Section 4, Figure 1): a total compilation from active trails to graph reachability, certified by node-survival proofs.
- (4) We mechanize the reverse witness extractor (Section 5, Figure 1): every stage is constructive, including `route_improves_of_bad`, which strictly decreases a well-founded measure.
- (5) We close the **full equivalence** (`dSeparated_iff_dSeparates`): on the pairwise-disjoint domain, moral-graph separation holds if and only if every trail is blocked, with both directions discharged by the optimizer and the decompiler.
- (6) We bridge the graph core to a Shannon leakage bound via a two-stage factorisation split (Section 3.4): a computable upper stage producing decide-able Markov-blanket conditions, and a typed lower stage delivering the probabilistic conditional-independence predicate `condMarkov`. Composing the split with a dual KL witness yields an upper bound for information leakage.

The graph results are available as a standalone Lean 4 package. The decompiler produces not a mere existence proof but a computational witness—a Σ -type containing the actual trail and the proof that it is unblocked.

2 Background and the Endpoint Discrepancy

2.1 Trail Blocking (Operational Semantics)

An undirected *trail* to y in a DAG G is a finite walk that may follow edges forward or backward. A consecutive triple $a \rightarrow b \rightarrow c$ or $a \leftarrow b \rightarrow c$ is a *non-collider*; $a \rightarrow b \leftarrow c$ is a *collider*. A triple is *blocked* by Z if:

- it is a non-collider whose middle node is in Z ; or
- it is a collider whose middle node and all descendants are disjoint from Z .

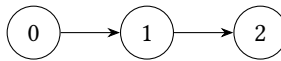
A trail is *blocked* by Z if some internal triple is blocked. Then Z *d-separates* X from Y in the trail sense if every trail from X to Y is blocked by Z .

2.2 Moral Graph Separation (Static Analysis)

The *ancestral subgraph* of S is the subgraph induced by all ancestors of nodes in S . Its *moral graph* is obtained by (1) forgetting edge directions and (2) connecting all co-parents (pairs of nodes with a common child). The *d-separation graph* is the moral graph of the ancestral subgraph of $X \cup Y \cup Z$ with Z removed. Then Z d-separates X from Y in the graph sense if no node of X is connected to any node of Y in this graph.

2.3 The Endpoint Discrepancy (Counterexample)

Our counterexample lives on the simplest non-trivial DAG:



Let $X = \{0\}$, $Y = \{1\}$, $Z = \{0\}$.¹

THEOREM 2.1 (ENDPOINT-IN- Z COUNTEREXAMPLE). *There exists a DAG G and finite sets $X, Y, Z \subseteq \mathbb{N}$ such that G d-separates X from Y in the moral-graph sense, but Z does not d-separate X from Y in the trail-blocking sense.*

¹Node 2 and the edge $1 \rightarrow 2$ represent a downstream path to illustrate a general chain, though the minimal discrepancy itself is exhibited directly on the edge $0 \rightarrow 1$.

Listing 1. Mechanized counterexample

```

theorem dsep_complete_endpoint_in_Z_counterexample :
  \exists (G : DAG) (X Y Z : Finset \N),
    DAG.dSeparated G X Y Z \wedge \neg dSeparates G X Y Z := by
  refine <chain3, {0}, {1}, {0}, ?_, ?_>
  ...

```

Why the disagreement happens. Trail blocking only inspects *internal* triples. The one-edge trail $0 \rightarrow 1$ has no internal triple at all, so nothing can block it. By contrast, the moral-graph construction deletes Z unconditionally; since $0 \in Z$, the edge $(0, 1)$ disappears from the moralized graph, and 0 and 1 are disconnected.

2.4 Textbook Equivalence and the Disjointness Repair

Standard references state that the two criteria are equivalent under the implicit assumption that X , Y , and Z are pairwise disjoint. In applications, however, the disjointness precondition is often omitted. As shown in the counterexample above (Section 2.3), this omission is not benign.

If we require $X \cap Z = \emptyset$ and $Y \cap Z = \emptyset$ (enforced by the safety invariant $\text{DisjointSets } X \ Y \ Z$), the start node of any trail is never deleted, and the endpoint caveat disappears.

3 A Three-Layer Architecture

Our formal development uses three layers mirroring a verified compiler’s architecture.

3.1 Layer 1: Type-Safe Graph Syntax

Nodes are natural numbers, edges are a finite set of ordered pairs ($\text{Finset } (\mathbb{N} \times \mathbb{N})$), and acyclicity is well-foundedness of the edge relation. The key safety invariant is $\text{DisjointSets } X \ Y \ Z$, which requires pairwise disjointness of the three node sets. This constraint can be conceptualized by analogy to substructural types as an ownership boundary: the conditioned variables Z must be disjoint from (and thus cannot alias) the source X or sink Y .

Listing 2. DAG structure and safety predicate

```

structure DAG where
  nodes : Finset \N
  edges : Finset (\N \times \N)
  edges_subset : \forall {u v}, (u,v) \in edges -> u \in nodes \wedge v \in nodes
  acyclic : WellFounded fun u v => (u,v) \in edges

def DisjointSets (X Y Z : Finset \N) : Prop :=
  Disjoint X Y \wedge Disjoint X Z \wedge Disjoint Y Z

```

3.2 Layer 2: Certified Intermediate Representations

Between operational trails and graph reachability sits a family of IRs that preserve just enough structure for certified translation.

MAGWalk. A compressed walk language equivalent to moral-graph reachability. It has three constructors: reflexivity, a single directed or reversed edge (MAGWalk.single), and a “moral jump” over an active collider (MAGWalk.jump). The collider node is skipped, which is why compression is correct.

StaticRoute. A type-valued decompiler IR that keeps the witness hidden by graph reachability. Each step is either a forward edge, a backward edge, or a moral jump storing the common child:

Listing 3. Static route IR (excerpt)

```
inductive StaticStep (G : DAG) (X Y Z : Finset \N) : \N -> \N -> Type where
| directForward (hEdge : G.HasEdge u v) ...
| directBackward (hEdge : G.HasEdge v u) ...
| moralJump (huw : G.HasEdge u w) (hvw : G.HasEdge v w)
  (hw : w \in G.ancestralSubgraphNodes (X \cup Y \cup Z)) ...
```

OpenTrace. The compiler target before converting to an active route. Each cons node stores its oriented DAG traversal *and* a local non-blocking proof, making *OpenTrace* a type-valued certificate that every junction is Z-open.

3.3 Layer 3: Witness-Extraction Pipelines

The two pipelines are summarized in Figure 1.

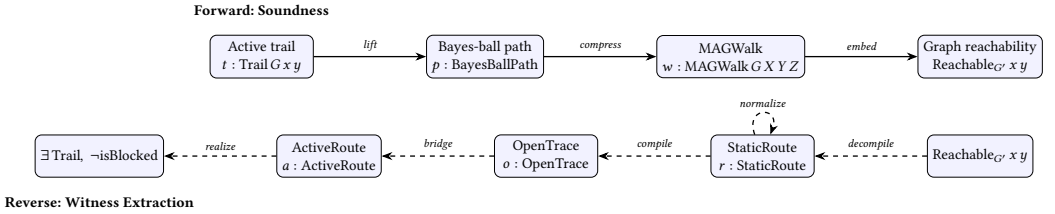


Fig. 1. The two witness-extraction pipelines. Solid arrows: forward (soundness). Dashed arrows: reverse (completeness). Every arrow, including the *normalize* step, is a total constructive function in Lean; the *normalize* step is a proved bad-collider minimization (Section 5).

3.4 Two-Stage Factorisation Split and QIF Bridge

The d-separation result supports a Shannon leakage-bound pipeline. Once the equivalence between moral-graph separation and trail blocking is established, we can replace a conditional-independence premise by a checked graph property in a cut-set capacity argument. The full finite chain is:

$$\text{d-separation} \Rightarrow \text{conditional independence} \Rightarrow \text{conditional DPI} \Rightarrow \text{leakage bound.}$$

To bridge from structural d-separation to quantitative leakage bounds, our architecture has two stages:

- **Upper stage.** Markov-blanket and local-Markov conditions for finite DAGs are decidable. The implementation is in `Graph/MarkovBlanket.lean`; the helpers are `computeMarkovBlanket`, `generateMarkovConditions`, and `generateMarkovBlanketConditions`. `chain3` and `collider3` are discharged by `decide` in `Examples/MarkovBlanket.lean`.
- **Lower stage (concrete-tuple adapter).** `CausalModel/Factorization.lean` converts those node-set premises into `condMarkov` via `condMarkovNodeCI` and the typed 4-variable adapter. `FactorizesOverDAG` captures the semantic joint without explicit row/column expansion.

The two stages compose: a topological proof obligation discharged by `decide` flows through the adapter to a probabilistic CI fact usable in the leakage chain.

Upper-bound certificate. On top of the split we prove a sufficiency theorem for leakage bounds via a dual KL witness. Given a witness distribution $\omega(b \mid c)$, `condMutualInfo_le_of_dual_witness` bounds the Shannon leakage $I(A; B \mid C)$ by the KL divergence between the joint and the factorized witness. The statement establishes sufficiency for a given witness and does not claim tightness or an optimizer.

4 Forward Soundness

Under `DisjointSets X Y Z`, we mechanize the soundness direction:

$$\text{DAG.dSeparated}(G, X, Y, Z) \implies \text{dSeparates}(G, X, Y, Z).$$

The proof follows a three-stage compilation pipeline.

Stage 1: Trail $\rightarrow$ Bayes-ball path. If a trail t from $x \in X$ to $y \in Y$ is not blocked by Z , and $x \notin Z$ (guaranteed by `Disjoint X Z`), then we lift t to an explicit Bayes-ball state path. The construction is by induction on the trail, analyzing each directional triple to show that the Bayes-ball state machine can traverse it.

Listing 4. Trail to Bayes-ball path

```
def bayesBallPath_of_active_trail_outOf
  (t : Trail G u v) (h_active : \neg t.isBlocked Z)
  (huZ : u \notin Z) :
  \Sigma final_dir, BayesBallPath G Z (u, outOf) (v, final_dir)
```

Stage 2: Certified membership. The Bayes-ball path contains intermediate states that must “survive” deletion of Z . We prove a certified version that guarantees every `RequiredState` node belongs to the d -separation graph nodes:

Listing 5. Certified Bayes-ball path

```
def bayesBallPathCert_of_active_trail_outOf
  (t : Trail G u v) (h_active : \neg t.isBlocked Z)
  (huZ : u \notin Z)
  (huA : u \in G.ancestralSubgraphNodes (X \cup Y \cup Z))
  (hvD : v \in G.dSeparationGraphNodes X Y Z) :
  \Sigma final_dir, {p : BayesBallPath ... //
    \forall {q}, BayesBallPath.RequiredState p q ->
      q.1 \in G.dSeparationGraphNodes X Y Z}
```

Stage 3: Compress to MAGWalk. A Bayes-ball path is a sequence of single-edge steps. The compress algorithm scans it with a two-step window: ordinary non-collider windows become a `MAGWalk.single`, while active-collider windows $(a, _) \rightarrow (b, \text{into}) \rightarrow (c, \text{outOf})$ become a single `MAGWalk.jump` over the collider b . The collider itself need not survive deletion of Z , which is why this compression is correct.

THEOREM 4.1 (SOUNDNESS). *For any DAG G and pairwise-disjoint finite sets $X, Y, Z \subseteq \mathbb{N}$, if G d -separates X from Y in the moral-graph sense, then every trail from X to Y is blocked by Z .*

Listing 6. Top-level soundness theorem

```
theorem dSeparated_of_dSeparated_disjoint
  (hXYZ : DisjointSets X Y Z)
  (hsep : DAG.dSeparated G X Y Z) : dSeparates G X Y Z
```

5 Reverse Witness Extraction

The reverse direction reconstructs an active-trail witness from $\neg \text{DAG.dSeparated}$, showing that the conditioning set Z fails to block the flow. The pipeline is:

$$\begin{aligned} \text{Reachable} &\rightarrow \text{StaticRoute} \rightarrow \text{NormalizedStaticRoute} \\ &\rightarrow \text{OpenTrace} \rightarrow \text{ActiveRoute} \rightarrow \exists \text{ Trail}, \neg \text{isBlocked}. \end{aligned}$$

5.1 From Reachability to StaticRoute

Graph reachability is a **Prop**-valued relation that hides the structure of each step. We introduce `StaticStep` and `StaticRoute` as type-valued evidence: a `StaticRoute` is an explicit list of directed edges, reversed edges, and moral jumps, each carrying the witness needed for later stages.

Listing 7. Reachability to static route

```
theorem nonemptyStaticRoute_of_dSeparationGraph_reachable
  (h : (G.dSeparationGraph X Y Z).Reachable u v) :
  Nonempty (StaticRoute G X Y Z u v)
```

5.2 Normalization: Eliminating Bad Colliders

A “bad collider” is a collider window whose middle node and all descendants are disjoint from Z . Such a collider is blocked, so it must be eliminated. Normalization may change endpoints: if a bad collider leaks ancestry into X or Y , we reroute the prefix or suffix to a different node in that set.

We package static routes as endpoint-carrying witnesses (`StaticRouteWitness`) and define a bad-collider count `routeBadCount`. The minimization is a `Nat.find` wrapper: if every nonzero-count witness can be strictly improved, a zero-count witness exists.

Listing 8. Bad-collider minimization wrapper

```
theorem normalized_route_exists_of_improves
  (hwit : \exists w : StaticRouteWitness G X Y Z, True)
  (himprove : \forall w, routeBadCount w \neq 0 ->
    \exists w', routeBadCount w' < routeBadCount w) :
  \exists w, routeBadCount w = 0
```

The improvement obligation is discharged by the core normalization theorem, which has no **sorry**:

Listing 9. Strict improvement for non-normalized routes

```
theorem route_improves_of_bad
  (w : StaticRouteWitness G X Y Z) (hbad : routeBadCount w \neq 0) :
  \exists w' : StaticRouteWitness G X Y Z,
  routeBadCount w' < routeBadCount w
```

The dispatcher proof extracts the first bad collider, exhibits an ancestral escape into X or Y , and reattaches a chain whose nodes all survive in the d-separation graph. Replacing one moral jump by zero-cost chain edges strictly decreases the bad-collider count, so the well-founded recursion bottoms out at zero bad colliders. Rerouting is permitted to land on a different node of X or Y .²

²The proof is structured via several constituent lemmas (`exists_split`, `ancestor_escape`, `bad_child_survives`, `escape_path_survives`, and corresponding count-reduction lemmas) implemented within the decompiler normalization module.

5.3 From Zero-Bad StaticRoute to OpenTrace to ActiveRoute

A static route with zero bad colliders compiles directly to an OpenTrace, because every junction is locally open:

Listing 10. Zero-bad compiler

```
theorem openTrace_of_countBadColliders_zero
  (hzero : countBadColliders arrival route = 0) :
  Nonempty (\Sigma finalDir, OpenTrace G Z (x, arrival) (y, finalDir))
```

The OpenTrace is then converted to a BayesBallPath and wrapped as an ActiveRoute. Finally, ActiveRoute.to_activeTrail yields the existential witness:

Listing 11. Final decompiler theorem

```
theorem activeWitness_of_not_dSeparated
  (hnot : \neg DAG.dSeparated G X Y Z) : ActiveWitness G X Y Z
```

The result is a constructive Σ -type containing the actual nodes $x \in X$, $y \in Y$, the final direction, and the active route.

5.4 The Closed Equivalence

Composing the optimizer (Section 4) and the decompiler yields the full equivalence on the well-typed domain—the bisimulation statement: on pairwise-disjoint X, Y, Z , the operational and static languages agree exactly.

Listing 12. Full d-separation equivalence

```
theorem dSeparated_iff_dSeparates
  (hXYZ : DisjointSets X Y Z) :
  DAG.dSeparated G X Y Z <=> dSeparates G X Y Z
```

REMARK 1 (VERIFICATION STATUS). *The lower stage takes FactorizesOverDAG as a caller-supplied hypothesis, representing the interface boundary between the topological upper stage and the probabilistic adapter. Given this interface, the mathematical chain from graph-semantic d-separation to the quantitative dual-witness leakage bound is machine-checked and contains no **sorry**. The final graph-layer obligation was discharged by reachable_equiv_reachableFinset, which identifies reachability in a finite DAG with the reflexive–transitive closure of its edge relation. The resulting Lean development comprises 158 theorems and 130 definitions type-checked without errors.*

6 Related Work

Our mechanization bridges causal inference, mechanized probabilistic semantics, and verified compilation.

Mechanized Causal Inference and D-Separation. Chancelier, De Lara, and Heymann (2024) [1] mechanized d-separation as a binary relation using Coq and MathComp/SSReflect, providing a relational-algebraic treatment of graphical separation. Parallel to our work, Anna Zhang (2025) [8] formalized the semantics of conditional independence and d-separation in Coq, bridging structural properties with function-based definitions for experimental design.

Both developments adopt standard mathematical formulations without exposing the implicit textbook domain caveats. Our work contributes a formalized counterexample and repair for the gap between syntactic trails and moralized node-deletion. We also move from purely relational

verification to a constructive equivalence that generates executable IRs (e.g., OpenTrace) in Lean 4 for witness synthesis.

Categorical and Topological D-Separation. Fritz and Klingler (2023) [2], extending work by Fritz and Liang (2023) [3], abstracted d-separation using string diagrams in Markov categories. Their framework bypasses moralization and reads conditional independence from topological connectedness in a generalized measure-theoretic space. Our approach is complementary: Markov categories provide a general categorical theory, while our artifact targets the legacy operational graph definitions (moral graphs and trace trails) used in existing software. By framing `DisjointSets` as a disjointness constraint, we show that classical d-separation behaves like a disjointness-constrained query syntax within a broader categorical structure.

Trace Bisimulation and Witness Extraction. The forward pipeline (soundness) translates active trails into a compressed step-relation (`MAGWalk`), in the spirit of semantics-preserving compiler passes such as `CompCert` (2009) [5]. The reverse pipeline (completeness) unwinds reachability into an active trail, resembling Proof-Carrying Code (1997) [6]: instead of returning a boolean assertion of an information leak, it synthesizes an explicit machine-checkable witness. This provides a graph-theoretic substrate for future verified static analyzers of quantitative information flow.

7 Conclusion

The textbook equivalence of trail blocking and moral-graph separation is not unconditional: a one-edge trail with an endpoint in Z is enough to break it. Mechanizing the repair as a set-theoretic disjointness precondition (`DisjointSets`) recovers the equivalence in both directions and makes the disjointness premise explicit. For verified causal-inference tooling the consequence is practical: any pipeline that consumes d-separation as a precondition for conditional independence must surface the disjointness premise as a checked side condition.

The result covers the structural half of a Shannon leakage chain whose connection to probability distributions is established conditional on factorization, defining the clean interface boundary between graph topology and measure theory. Future work targets: (1) formalizing concrete numeric probability mass functions to exercise the verification chain end-to-end; (2) transitioning from custom graph logic imports to targeted `mathlib` theorem structures; (3) lifting the probabilistic foundations from discrete finite distributions to general measure-theoretic spaces; and (4) mechanizing the first-order surface syntax, rank checker, and query safety guarantees.

References

- [1] Jean-Philippe Chancelier, Michel De Lara, and Benjamin Heymann. 2024. Conditional Separation as a Binary Relation: A Coq Assisted Proof. Preprint. arXiv:2404.01777. Extended version available at hal-03315809.
- [2] Tobias Fritz and Andreas Klingler. 2023. The d-Separation Criterion in Categorical Probability. *Journal of Machine Learning Research (JMLR)* 24, 46 (2023), 1–49.
- [3] Tobias Fritz and Wendong Liang. 2023. Free gs-Monoidal Categories and Free Markov Categories. *Applied Categorical Structures* 31, 2 (2023), 21.
- [4] Steffen L. Lauritzen. 1996. *Graphical Models*. Oxford University Press, Oxford, UK.
- [5] Xavier Leroy. 2009. Formal verification of a realistic compiler. *Commun. ACM* 52, 7 (2009), 107–115.
- [6] George C. Necula. 1997. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, New York, NY, USA, 106–119.
- [7] Judea Pearl. 2009. *Causality* (2nd ed.). Cambridge University Press, Cambridge, UK.
- [8] Anna Zhang. 2025. *Formalizing Causal Models Through the Semantics of Conditional Independence*. Master’s thesis. Massachusetts Institute of Technology. Supervised by Adam Chlipala.